

3.1 Waktu Pelaksanaan Praktikum

Durasi kegiatan praktikum = **170 menit**, dengan pembagian waktu sebagai berikut:

- **50 menit pertama** → Mahasiswa mengerjakan latihan coding Compose UI.
- **50 menit kedua** → Pembahasan latihan coding bersama asisten.
- **70 menit terakhir** → Pengayaan materi, diskusi, dan pengerjaan tugas akhir bab. **3.2**

Tujuan

Setelah mengikuti praktikum ini, mahasiswa diharapkan dapat:

1. Memahami konsep **state** sebagai sumber kebenaran (*single source of truth*) dalam Jetpack Compose.
2. Menggunakan **state lokal** dengan `remember { mutableStateOf() }` untuk membuat tampilan yang dapat berubah secara dinamis.
3. Menjelaskan bagaimana perubahan nilai state secara otomatis memicu **recomposition** pada UI.
4. Menghubungkan state dengan **komponen UI dasar** (Text, Button, Image) sehingga UI lebih interaktif.
5. Membiasakan diri memisahkan data (state) dan tampilan (UI) agar aplikasi lebih mudah dipelihara.

3.3 Alat dan Bahan

Untuk melaksanakan praktikum ini, mahasiswa memerlukan:

1. Perangkat keras (hardware):

- Laptop/PC dengan RAM minimal 8 GB (disarankan 16 GB).
- Prosesor minimal Intel i5 generasi ke-8 atau setara (AMD Ryzen 5).
- Ruang penyimpanan kosong minimal 10 GB untuk SDK dan emulator.

Perangkat lunak (software):

- **Android Studio** (versi stabil terbaru).

- **Android SDK** (minimal API 24 / Android 7.0, disarankan API 33+).
- Emulator Android (AVD) **atau** perangkat Android fisik dengan USB debugging aktif.

3. Project awal:

- Buat project baru dengan template **Empty Activity (Jetpack Compose)**.
- Gunakan nama project: **Praktikum3_NIMNama**.

3.4 Dasar Teori

3.4.1 UI Deklaratif dan Peran State

- Jetpack Compose menggunakan paradigma **UI deklaratif**: UI adalah representasi langsung dari **state** aplikasi.
- **State** adalah data yang mengontrol tampilan. Jika state berubah → UI otomatis *recompose* sesuai data terbaru.
- Tanpa state, UI akan statis dan tidak dapat beradaptasi dengan perubahan interaksi pengguna.

3.4.2 State di Jetpack Compose

- State didefinisikan dengan **remember { mutableStateOf() }**.
- **remember** menjaga nilai state tetap ada selama lifecycle Composable.
- **mutableStateOf** adalah *observable holder* yang memberi tahu Compose kapan data berubah.

Contoh:

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }
    Button(onClick = { count++ }) {
        Text("Klik: $count")
    }
}
```

3.4.3 State Hoisting (Pengangkatan State)

- *State hoisting* adalah praktik **memisahkan state dari UI**.
- Tujuannya agar **Composable lebih bersih, reusable, dan mudah diuji**.
- Prinsip: *UI menerima data (state) dan callback (event handler), bukan menyimpan data sendiri.*

Contoh tanpa hoisting (kurang baik):

```
@Composable
fun FollowButton() {
    var isFollowed by remember { mutableStateOf(false) }
    Button(onClick = { isFollowed = !isFollowed }) {
        Text(if (isFollowed) "Unfollow" else "Follow")
    }
}
```

Contoh dengan hoisting (lebih baik):

```
@Composable
fun FollowButton(
    isFollowed: Boolean,
    onClick: () -> Unit
) {
    Button(onClick = onClick) {
        Text(if (isFollowed) "Unfollow" else "Follow")
    }
}
```

Pemanggilan di Activity/Composable induk:

```
var isFollowed by remember { mutableStateOf(false) }
FollowButton(isFollowed = isFollowed, onClick = { isFollowed = !isFollowed })
```

3.4.4 State dan Recomposition

- **Recomposition** adalah proses Compose menggambar ulang UI ketika state berubah.
- Compose hanya menggambar ulang bagian yang relevan, bukan seluruh layar. •

Prinsip penting: *State → UI, bukan UI → State.*

3.4.5 Contoh Kasus: Toggle Button

- Mahasiswa dapat mencoba membuat tombol **Follow/Unfollow**:
 - Saat pertama kali aplikasi dijalankan → tombol bertuliskan "Follow".
 - Setelah diklik → berubah menjadi "Unfollow".
 - Klik lagi → kembali ke "Follow".

Kode:

```
@Composable
fun ToggleButton() {
    var isOn by remember { mutableStateOf(false) }
    Button(onClick = { isOn = !isOn }) {
        Text(if (isOn) "Unfollow" else "Follow")
    }
}
```

3.5 Prosedur/Pelaksanaan Praktikum

3.5.1 Membuat Project Baru

1. Buka **Android Studio** → pilih **New Project**.
2. Pilih template **Empty Activity (Jetpack Compose)**.
3. Isi nama aplikasi: **Praktikum3_State_NIMNama**.
4. Klik **Finish**, tunggu Gradle selesai sinkronisasi.

3.5.2 Menambahkan State Sederhana (Counter)

1. Buka file **MainActivity.kt**.
2. Tambahkan Composable berikut:

```

@Composable
fun CounterApp() {
    var count by remember { mutableStateOf(0) }
    Column(
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize().padding(16.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text("Jumlah Klik: $count")
        Spacer(modifier = Modifier.height(8.dp))
        Button(onClick = { count++ }) {
            Text("Tambah")
        }
    }
}

```

3. Panggil di dalam `setContent`:

```

setContent {
    CounterApp()
}

```

4. Jalankan aplikasi → tekan tombol beberapa kali, amati teks berubah. **3.5.3**

Percobaan dengan Toggle State

1. Tambahkan Composable untuk tombol Follow/Unfollow:

```

@Composable
fun FollowButton() {
    var isFollowed by remember { mutableStateOf(false) }
    Button(onClick = { isFollowed = !isFollowed }) {
        Text(if (isFollowed) "Unfollow" else "Follow")
    }
}

```

2. Panggil `FollowButton()` di `setContent`.

3. Jalankan aplikasi → klik tombol untuk melihat perubahan teks sesuai state.

3.5.4 State Hoisting (Pengangkatan State)

1. Modifikasi kode agar **state tidak disimpan langsung di Composable**.

```
@Composable
fun FollowButton(
    isFollowed: Boolean,
    onClick: () -> Unit
) {
    Button(onClick = onClick) {
        Text(if (isFollowed) "Unfollow" else "Follow")
    }
}
```

2.

Kelola state di level atas (**MainActivity** atau Composable induk):

```
@Composable
fun FollowApp() {
    var isFollowed by remember { mutableStateOf(false) }
    FollowButton(
        isFollowed = isFollowed,
        onClick = { isFollowed = !isFollowed }
    )
}
```

3. Panggil **FollowApp()** di **setContent**.

4. Jalankan aplikasi → pastikan toggle tetap berfungsi, meski state dipisahkan dari UI. **3.5.5 Studi**

Kasus Mini (Profil dengan State)

1. Susun tampilan **profil sederhana**:

- Foto profil (Image dari **drawable**).
- Nama mahasiswa (Text).
- Tombol **Follow/Unfollow** dengan state.

2. Contoh kode:

```
@Composable
fun ProfileCard() {
```

```

var isFollowed by remember { mutableStateOf(false) }

Column(

    horizontalAlignment = Alignment.CenterHorizontally,
modifier = Modifier

    .fillMaxSize()

    .padding(16.dp),
    verticalArrangement = Arrangement.Center

) {

    Image(

        painter = painterResource(R.drawable.profil),
contentDescription = "Foto Profil",

        modifier = Modifier.size(120.dp).clip(CircleShape)    )

    Spacer(modifier = Modifier.height(8.dp))

    Text("Nama: Andi", fontSize = 20.sp, fontWeight = FontWeight.Bold)

    Spacer(modifier = Modifier.height(4.dp))

    Text("Mahasiswa Teknik Informatika")

    Spacer(modifier = Modifier.height(12.dp))

    Button(onClick = { isFollowed = !isFollowed }) { Text(if
(isFollowed) "Unfollow" else "Follow")    }

}

}

```

3.6 Data dan Analisis Hasil Percobaan

Jawablah pertanyaan berikut berdasarkan percobaan yang sudah dilakukan. Tuliskan jawaban di laporan praktikum.

1. State Dasar

- Apa yang terjadi pada tampilan ketika nilai `count` diubah pada aplikasi Counter?
- Mengapa teks pada Button dan Text bisa langsung berubah tanpa dipanggil ulang secara manual?

2. Toggle State

- Bagaimana Compose mengetahui kapan harus mengganti teks "Follow" menjadi "Unfollow"?
- Apa yang akan terjadi jika variabel `isFollowed` dideklarasikan sebagai `var` biasa (tanpa `remember { mutableStateOf() }`)?

3. State Hoisting

- Jelaskan perbedaan pendekatan **tanpa state hoisting** dan **dengan state hoisting**.
- Mengapa state hoisting membuat kode lebih bersih dan mudah digunakan kembali (*reusable*)?

4. Modifier + State

- Bagaimana Modifier (`background`, `clickable`) bekerja bersama state untuk mengubah warna kotak?
- Apakah UI tetap responsif jika Modifier hanya digunakan tanpa state?

5. Studi Kasus Profil

- Bagaimana tombol Follow/Unfollow di dalam `ProfileCard` memanfaatkan state untuk mengubah teks?
- Jika aplikasi ditutup dan dibuka lagi, apakah state masih tersimpan? Mengapa demikian?

3.7 Tugas Praktikum

Untuk memperkuat pemahaman mahasiswa tentang **state di Jetpack Compose**, kerjakan tugas berikut:

1. Aplikasi Counter Plus-Minus

- Buat aplikasi dengan dua tombol: **Tambah (+)** dan **Kurang (-)**.
- Gunakan state (`remember { mutableStateOf() }`) untuk menyimpan nilai counter.
- Pastikan nilai counter tidak bisa turun di bawah nol.

2. Tombol Toggle Warna

- Buat sebuah **Box** berukuran 200×200 dp.
- Klik pertama → warna **Merah**.
- Klik kedua → warna **Hijau**.
- Klik berikutnya → bergantian kembali.
- Gunakan state untuk menyimpan warna saat ini.

3. Aplikasi Profil Interaktif

- Tampilkan foto profil (dummy), nama, dan deskripsi singkat.
- Tambahkan tombol **Follow/Unfollow** yang berubah sesuai state.
- Tambahkan indikator teks di bawah tombol, contoh:
 - Jika diikuti → tampilkan “Anda mengikuti akun ini”.
 - Jika tidak diikuti → tampilkan “Anda belum mengikuti akun ini”.

4. Output yang Dikumpulkan

- File `MainActivity.kt` berisi kode utama.
- Screenshot aplikasi saat dijalankan (minimal 2 kondisi berbeda).
- File `README.md` berisi:
 - Penjelasan singkat implementasi state di aplikasi.
 - Analisis singkat: mengapa UI Compose lebih sederhana dibandingkan XML tradisional untuk kasus ini.